SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I _____M Vineela(192371053) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

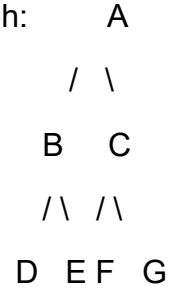
Signature of the student: *Vineela*

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

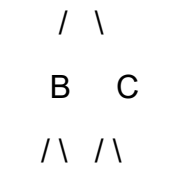
Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

A



Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. Initial State :

1 3 6

5 0 2

4 7 8

Goal State :

1 2 3

4 5 6

7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

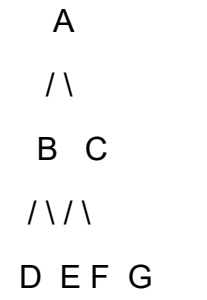
Criteria	**Weightage (%)**	**Level 4 – Exemplary (4)**	**Level 3 – Proficient (3)**	**Level 2 – Developing (2)**	**Level 1 – Beginning (1)**
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Annexure A – Dry Run Evaluation

Question 1: Breadth-First Search (BFS)

Given graph



BFS (Breadth-First Search) visits nodes level by level. It visits the root first, then its children, and then the next level.

BFS uses a Queue (FIFO), which means First In, First Out.

Step-by-step execution

1. Start at node A and visit it. Add its children B and C to the queue.
2. Remove B from the queue and visit it. Add D and E.
3. Remove C and visit it. Add F and G.
4. Remove D and visit it. It has no children.
5. Remove E and visit it. It has no children.
6. Remove F and visit it. It has no children.
7. Remove G and visit it. It has no children. The queue is now empty.

Completed table

Iteration	Queue after iteration	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	Empty	G

Answers

1. Write the BFS traversal order.

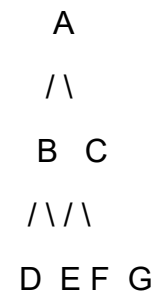
A → B → C → D → E → F → G

2. Show the queue contents after each iteration.

The queue contents are shown in the table above. For example, after visiting B, the queue is C, D, E because C was already waiting, and D and E were added at the end.

Question 2: Depth-First Search (DFS)

Use the same graph:



DFS (Depth-First Search) follows one branch as deeply as possible before returning to explore another branch.

DFS can use a Stack (LIFO), which means Last In, First Out. Here, we assume the left child is visited before the right child.

Step-by-step execution

1. Start at A.
2. Move left to B.
3. Move left to D. D has no children, so return to B.
4. Visit E, the remaining child of B.
5. Return to A and move right to C.
6. Visit F, the left child of C.
7. Visit G, the right child of C.

Completed table

Iteration	Visited Node	Explanation
1	A	Start at the root
2	B	Move to the left child
3	D	Explore the left branch deeply
4	E	Return and visit B's remaining child

5	C	Return to A and move right
6	F	Visit C's left child
7	G	Visit C's right child

DFS traversal order: A → B → D → E → C → F → G

Important difference: BFS explores level by level, whereas DFS explores depth by depth.

Question 3: 8-Puzzle Problem Using BFS

Given initial and goal states

Initial state:

1 3 6

5 0 2

4 7 8

Goal state:

1 2 3

4 5 6

7 8 0

Here, 0 represents the blank space. The objective is to arrange the tiles into the goal state by moving the blank space up, down, left, or right.

How BFS works in the 8-puzzle

1. Place the initial state in the queue.
2. Remove the state at the front of the queue.
3. Generate all valid states by moving the blank space.
4. Add new, unvisited states to the back of the queue.
5. Repeat until the goal state is found.

For the table below, we use the move order Up → Down → Left → Right and avoid revisiting previously discovered states.

First five iterations

For readability, each state is represented as three rows separated by /.

Iteration	Current State	Queue Before Expansion	Queue After Expansion

1	136/502/478	[S0]	[S1,S2,S3,S4]
2	106/532/478	[S1,S2,S3,S4]	[S2,S3,S4,S5,S6]
3	136/572/408	[S2,S3,S4,S5,S6]	[S3,S4,S5,S6,S7,S8]
4	136/052/478	[S3,S4,S5,S6,S7,S8]	[S4,S5,S6,S7,S8,S9,S10]
5	136/520/478	[S4,S5,S6,S7,S8,S9,S10]	[S5,S6,S7,S8,S9,S10,S11,S12]

State key:

- S0 = Initial state
- S1 = 106/532/478
- S2 = 136/572/408
- S3 = 136/052/478
- S4 = 136/520/478

The remaining state labels represent the newly generated states. This table shows only the first five expansions; BFS continues beyond these iterations until it reaches the goal.

1. Which state is expanded in each iteration?

The state at the front of the queue is expanded in each iteration. BFS then adds its valid, undiscovered neighboring states to the back of the queue.

2. How many states are generated in total?

The shortest solution path contains 9 states, including the initial and goal states. However, the total number of states generated by BFS can be greater because it explores other possible states before reaching the goal. The exact total depends on the move order and when the search stops.

3. Write the complete solution path

The shortest solution requires 8 moves:

Step	Blank move	Resulting state
0	Initial	136/502/478
1	Right	136/520/478
2	Up	130/526/478
3	Left	103/526/478

4	Down	123/506/478
5	Left	123/056/478
6	Down	123/456/078
7	Right	123/456/708
8	Right	123/456/780

The final state 123/456/780 is the goal state.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

BFS explores states level by level. It checks all states requiring one move before states requiring two moves, and so on. Since each move has the same cost, the first goal state reached has the minimum number of moves.